

DATA STRUCTURE AND ALGORITHM

กลยฺุทธทางอัลกอริธึม

รศ.นชิรัตน์ ราชบุรี
รหัสวิชา 04-622-201

อัลกอริธึมแบบใช้แรง (Brute Force Algorithm)

การแก้ปัญหาแบบที่เรียบง่ายที่สุด น่าจะเป็นการแก้ปัญหาแบบใช้แรง คำว่าใช้แรง หมายความว่า เราจะแก้ปัญหาโดยใช้กระบวนการในการการแก้ปัญหาให้ทำงานไปเรื่อยๆ และหาคำตอบที่มีความเป็นไปได้ทั้งหมดของปัญหา วิธีการแก้ปัญหาแบบใช้แรงนี้ อาจจะเหมือนว่าไม่มีประสิทธิภาพ เนื่องจากไม่มีความซับซ้อน หรือมีการจัดการกับปัญหาอย่างเป็นระเบียบ แต่จากแนวความคิดที่ ง่ายๆ ในการแก้ปัญหาทำให้ การเขียนโปรแกรมเพื่อแก้ปัญหานั้น สามารถเขียนได้อย่างรวดเร็ว ใช้เวลาไม่นานก็เขียนสำเร็จ โดยไม่ต้องคำนึงถึงประสิทธิภาพมากนัก

อัลกอริธึมแบบใช้แรง (Brute Force Algorithm)

การแก้ปัญหาแบบใช้แรงนี้เนื่องจากการแก้ปัญหาที่ง่าย ๆ จึงเหมาะสมกับปัญหาที่มีขนาดเล็กไม่ใหญ่ เพราะการแก้ปัญหา โดยวิธีใช้แรงไม่ต้องเสียเวลาในการวิเคราะห์ อัลกอริธึมว่ามีประสิทธิภาพมากน้อยเพียงใด หากใช้เวลาในการคิดอัลกอริธึมที่ดีมีประสิทธิภาพที่สุดในการแก้ปัญหา และเขียนโปรแกรมที่ยุ่งยากซับซ้อนในการทำงาน อาจจะใช้เวลามากกว่า การเขียนอัลกอริธึมแก้ปัญหาโดยวิธีง่าย ๆ เขียนโปรแกรมได้อย่างรวดเร็ว แต่มีระยะเวลาที่ใช้ในการได้ผลลัพธ์ ไม่แตกต่างกัน เนื่องจากปัญหามีขนาดเล็กไม่ใหญ่ แต่ต้องมั่นใจว่าปัญหาที่จะแก้ นั้นเป็นปัญหาที่เล็กจริงๆ

ตัวอย่างการแก้ปัญหาการเรียงลำดับข้อมูลแบบใช้แรง

การเรียงลำดับข้อมูลเป็นปัญหาหนึ่งที่ต้องมีการจัดการกับข้อมูลอย่างมีประสิทธิภาพ โดยที่ประสิทธิภาพในการประมวลผลนั้น ขึ้นอยู่กับอัลกอริธึมที่เลือกมาใช้ในการเรียงข้อมูล การเรียงข้อมูลมีอยู่หลากหลายวิธีการ มีประสิทธิภาพต่างกัน มีตั้งแต่ $O(n^2)$ จนถึง $O(n \log n)$ อย่างเช่นวิธีการเรียงข้อมูลแบบ เลือก (Selection sort) หรือการเรียงข้อมูลแบบฟอง (Bubble sort) เป็นวิธีการเรียงข้อมูลที่ลำดับการการเจริญเติบโตของข้อมูลเป็น $O(n^2)$ ซึ่งเป็นวิธีการแก้ปัญหาแบบใช้แรง ในขณะที่ การเรียงข้อมูลแบบเร็ว (Quick sort) มีลำดับการการเจริญเติบโตของข้อมูลเป็น $O(n \log n)$

ตัวอย่างการแก้ปัญหาการเรียงลำดับข้อมูลแบบใช้แรง

หากข้อมูลที่ต้องการเรียงนั้นมีจำนวนน้อย ก็ควรจะเลือกใช้วิธีการเรียงข้อมูลแบบเลือก หรือแบบฟอง ถึงแม้ว่าจะมีประสิทธิภาพไม่ดีนัก แต่เวลาที่ได้จากการประมวลผลไม่แตกต่างกับการเรียงข้อมูลแบบเร็ว นอกจากนั้นแล้ว เวลาที่ใช้ในการสร้างโปรแกรมยังน้อยอีกด้วยเนื่องจากไม่มีความซับซ้อนของวิธีการ

อัลกอริธึมแบบแบ่งแยกและเอาชนะ (Divide and Conquer)

วิธีการแก้ปัญหแบบแบ่งแยกและเอาชนะ จะใช้วิธีการแบ่งปัญหาออกเป็น ส่วนย่อยๆ แล้วทำการแก้ปัญหในส่วนย่อยนั้นที่น่าจะหาคำตอบออกมาได้ ซึ่งจะทำให้ ขอบเขตของปัญหานั้นแคบลง แต่หากยังไม่สามารถสรุปคำตอบได้อีก ก็จะทำ การแบ่งปัญหานั้นย่อยลงไปอีก จนกว่าจะสรุปคำตอบได้ โดยใช้ อัลกอริธึมเดิมในการแตกปัญหาออกเป็น ส่วนย่อยๆ จนกระทั่งสามารถสรุปคำตอบได้

ตัวอย่างการใช้งานวิธีการแบ่งแยกและเอาชนะ

ปัญหาหนึ่งที่ใช้วิธีการแบ่งแยกและเอาชนะคือการค้นหาข้อมูลแบบไบนารี ซึ่งจะใช้วิธีการแบ่งข้อมูลออกทีละครึ่ง เพื่อค้นหาข้อมูลที่ต้องการโดยการเปรียบเทียบข้อมูลที่ต้องการค้นหากับข้อมูลในขอบเขตที่กำหนด แล้วแบ่งครึ่งข้อมูลไปเรื่อยๆ จนกว่าจะพบข้อมูล หรือไม่พบข้อมูลเมื่อไม่สามารถแบ่งได้อีกแล้ว

การแก้ปัญหาก็อีกวิธีการหนึ่งที่น่าเอาหลักการแบ่งแยกและเอาชนะไปใช้คือวิธีการเรียงข้อมูลแบบผสาน (Merge sort) ซึ่งจะทำการแบ่งชุดข้อมูลออกมาเป็นสองส่วน แล้วแบ่งครึ่งแต่ละส่วนไปเรื่อยๆ จนไม่สามารถแบ่งครึ่งข้อมูลได้แล้ว จึงทำการเรียงข้อมูลรวมเข้ามาทีละคู่ จนมารวมเป็นข้อมูลชุดเดิมทั้งหมด ที่เรียงลำดับเรียบร้อยแล้ว

ตัวอย่างการใช้งานวิธีการแบ่งแยกและเอาชนะ

อัลกอริธึมการเรียงข้อมูลแบบผสาน

1. แบ่งกลุ่มข้อมูลจนไม่สามารถแบ่งได้

2. ทำซ้ำ

2.1 เปรียบข้อมูลระหว่างแต่ละชุด ตามตัวตำแหน่งตัวชี้ ซึ่งตำแหน่งตัวชี้ของแต่ละชุดจะชี้ไปยังตำแหน่งแรกเพราะเป็นข้อมูลที่เรียงลำดับมาแล้ว

2.2 นำข้อมูลที่น้อยที่สุดไปเก็บไว้ และเลื่อน ตำแหน่งตัวชี้ไปยังข้อมูลตัวถัดไปในชุดที่ถูกนำข้อมูลออกไป

อัลกอริธึมแบบย้อนรอย (Backtracking)

อัลกอริธึมแบบย้อนรอยเป็นวิธีการที่จะหาเส้นทางที่เป็นไปได้ทั้งหมดในการแก้ปัญหา แล้วพิจารณาไปที่ละเส้นทาง โดยที่ก่อนจะเข้าไปในแต่ละเส้นทางนั้น จะมีการพิจารณาว่าเส้นทางที่กำลังจะเข้าไปนั้น สามารถตัดทิ้งได้หรือไม่โดยไม่จำเป็นต้องเข้าไปหาผลลัพธ์ เนื่องจากไม่มีคำตอบที่ต้องการอยู่ หากเส้นทางที่เลือกเข้าไปแก้ปัญหา ไม่สามารถแก้ปัญหาหรือไม่มีคำตอบที่ต้องการ จะทำการย้อนกลับไปเลือกเส้นทางในการแก้ปัญหาเส้นทางอื่น จนกว่าจะพบการแก้ไขปัญหา หรือคำตอบที่ถูกต้อง

ตัวอย่างการใช้งานการแก้ปัญหาแบบย้อนรอย

การแก้ปัญหา n ควีน (n Queen)

การวางควีนจำนวน n ตัวลงในกระดานหมากรุกขนาด $n \times n$ โดยควีนทั้งหมดต้องไม่สามารถ “กิน” ควีนตัวอื่นได้ โดยควีนของหมากรุกสามารถ “กิน” หมากรุกหรือควีนตัวอื่นได้ หากหมากตัวนั้นอยู่ใน

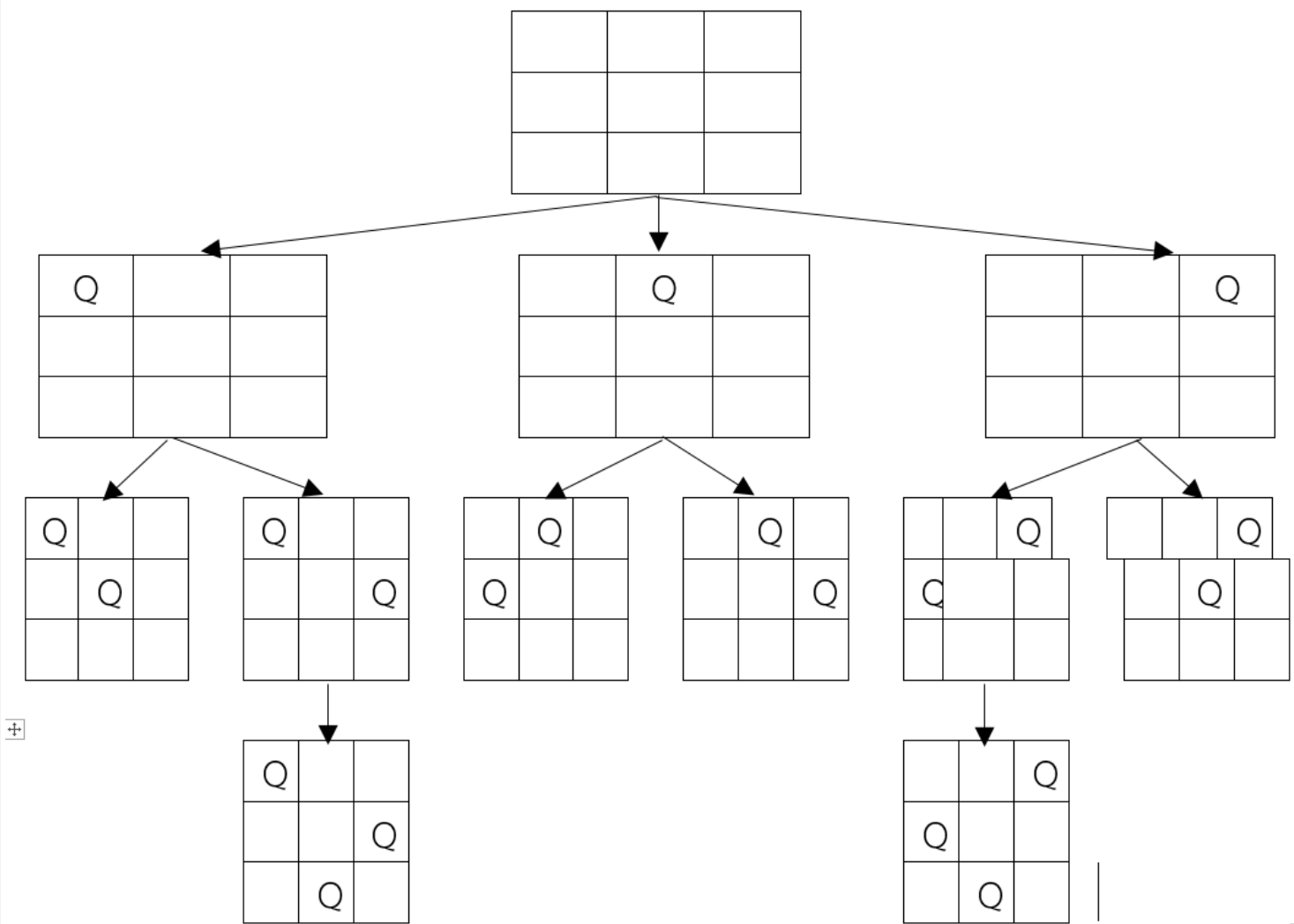
1. แถวเดียวกัน
2. คอลัมน์เดียวกัน
3. แนวเส้นทแยงมุมเดียวกัน (แนวเส้นทแยงมุมมีสองแนว)

ตัวอย่างการใช้งานการแก้ปัญหาแบบย้อนรอย

ขั้นตอนวิธี การวางควีน n ตัวลงในกระดานขนาด $n \times n$

1. กำหนดให้สแตกเป็นสแตกว่าง
2. เก็บข้อมูลตำแหน่งของควีนและวางควีนลงบนกระดานในคอลัมน์ที่ 1
3. วางควีน ตัวถัดไปลงบนกระดานในคอลัมน์และแถวถัดไป โดยไม่ต้องวางในคอลัมน์เดียวกันและโดยไม่กินกัน
4. หากมีการกินกันระหว่างควีน ให้ทำการย้อนกลับไปตำแหน่งเดิมของควีนตัวก่อนหน้าโดยการนำข้อมูลออกจากสแตก แล้วทำดำเนินการซ้ำเดิมในข้อที่ 2

ตัวอย่างการใช้งานการแก้ปัญหาแบบย้อนรอย



อัลกอริธึมเชิงละโมภ Greedy Algorithm

อัลกอริธึมสำหรับปัญหาประเภท optimization หลาย ๆ ข้อ มักจะประกอบด้วยหลายขั้นตอน ซึ่งแต่ละขั้นตอนจะต้องมีการตัดสินใจบางอย่าง

Greedy Algorithm คือประเภทของอัลกอริธึมที่มีแนวคิดที่ว่าใน แต่ละขั้นตอน จะตัดสินใจเลือกสิ่งที่ดีเหมือนเป็นสิ่งที่ดีที่สุดในตอนนั้นๆ เสมอ

อัลกอริธึมเชิงละโมภ Greedy Algorithm

- สำหรับบางปัญหา Greedy Algorithms จะให้คำตอบสุดท้าย ที่ดีที่สุดจริง ๆ (Globally optimal solution).
- แต่หลาย ๆ ปัญหา การใช้ Greedy Algorithms จะไม่ได้ คำตอบที่ดีที่สุด

อัลกอริธึมเชิงละโมภ Greedy Algorithm

□ ข้อดี:

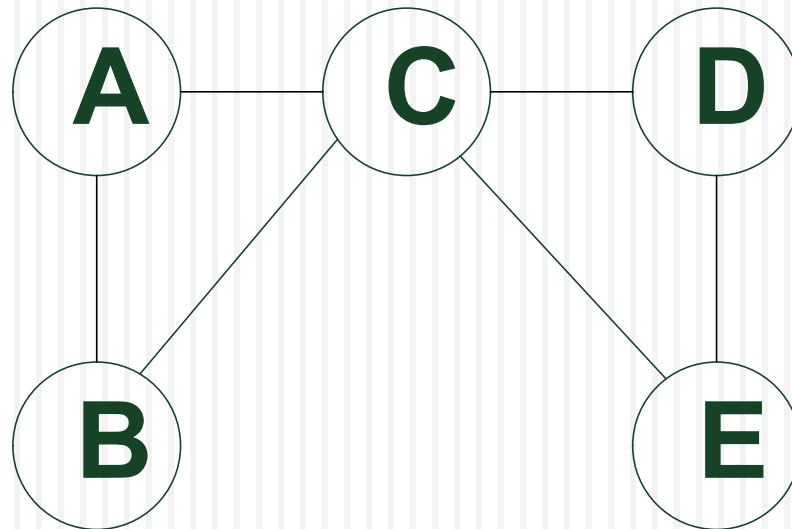
- มักจะง่ายในการเขียนโปรแกรม
- โปรแกรมมักจะมี Time Complexity ต่ำ เพราะสิ่งที่เลือกไปแล้ว มักจะไม่กลับมาเปลี่ยนการตัดสินใจทีหลัง

□ ข้อเสีย:

- มักไม่ได้ global optimal solution ในหลาย ๆ ปัญหา

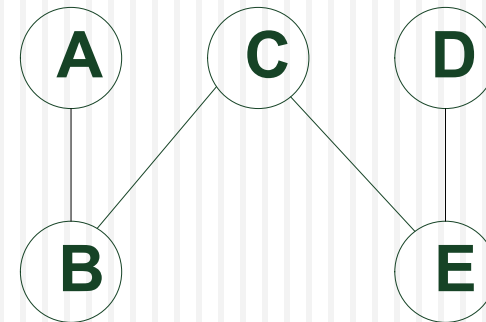
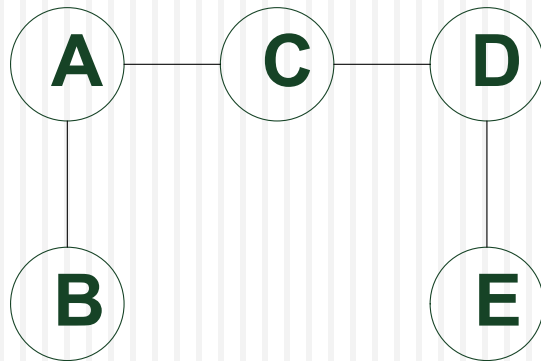
อัลกอริธึมแบบต้นไม้ทอดข้ามน้อยที่สุด (Minimum spanning tree)

ต้นไม้ทอดข้าม (Spanning Tree) เป็นต้นไม้ที่ถูกสร้างขึ้นมาจากกราฟ เนื่องจากต้นไม้ก็เป็นกราฟชนิดหนึ่ง โดยต้นไม้ทอดข้ามที่สร้างมา จะต้องมียุคยอดอยู่ครบทุกจุดยอดจากกราฟเดิม และ ไม่มี วงจรปิด (Close loop) เกิดขึ้นในต้นไม้



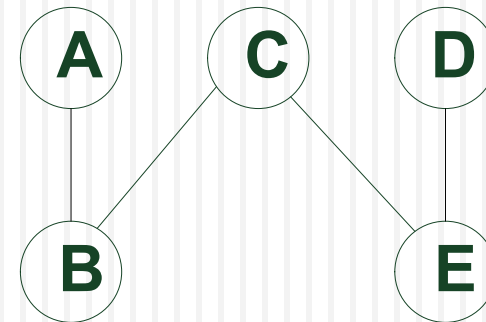
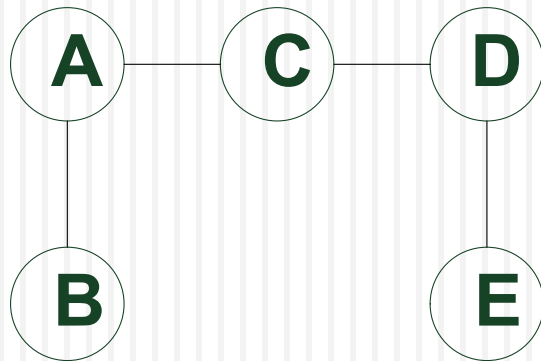
อัลกอริธึมแบบต้นไม้ทอดข้ามน้อยที่สุด (Minimum spanning tree)

จากรูปกราฟตัวอย่างสามารถสร้างต้นไม้ทอดข้ามได้หลากหลายรูปแบบเช่น



อัลกอริธึมแบบต้นไม้ทอดข้ามน้อยที่สุด (Minimum spanning tree)

จากรูปกราฟตัวอย่างสามารถสร้างต้นไม้ทอดข้ามได้หลากหลายรูปแบบเช่น

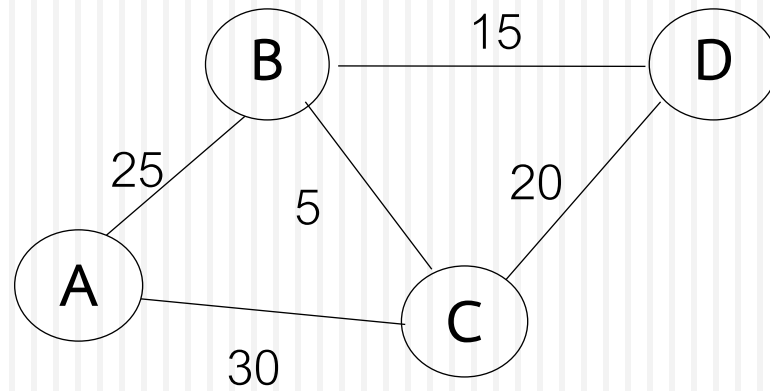


อัลกอริธึมของครุสคัล (Kruskal's Algorithm)

อัลกอริธึมของครุสคัลเป็นอัลกอริธึมที่หาต้นไม้ทอดข้ามน้อยสุดเช่นกัน โดยที่ใช้คิวแบบมีลำดับความสำคัญมาช่วยในการทำงาน โดยเก็บข้อมูลเส้นทางตามลำดับความสำคัญลงไปในคิว ข้อมูลที่มีค่าน้ำหนักน้อยคือ มีความสำคัญมาก จากนั้นเลือกเส้นทางออกมาจากคิวที่ละลำดับ แล้วตรวจสอบว่าเกิดวงจรปิด (Close loop) เกิดขึ้นหรือไม่ หากมีวงจรปิดเกิดขึ้นจะยกเลิกเส้นทางนั้น แล้วเลือกเส้นทางใหม่ทางคิวแบบมีลำดับความสำคัญต่อไป

อัลกอริธึมของครุสคัล (Kruskal's Algorithm)

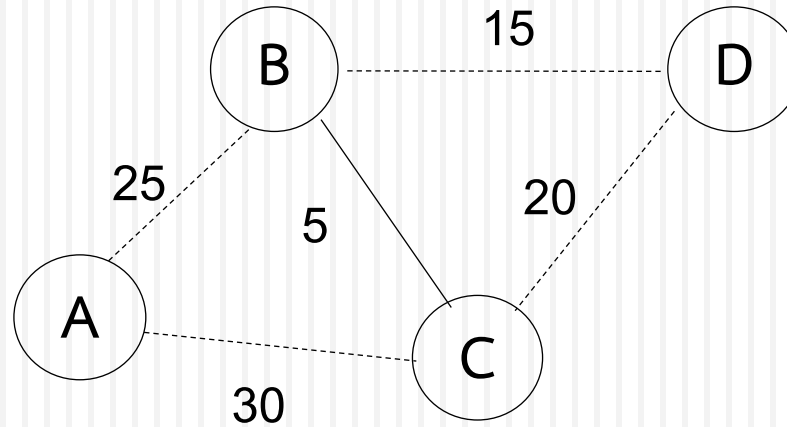
1. กำหนดให้จัดเก็บข้อมูลน้ำหนักตามลำดับความสำคัญลงไปในคิวแบบมีลำดับความสำคัญ



$PQ = \{(BC, 5), (BD, 15), (CD, 20), (AB, 25), (AC, 30)\}$

อัลกอริทึมของครุสคัล (Kruskal's Algorithm)

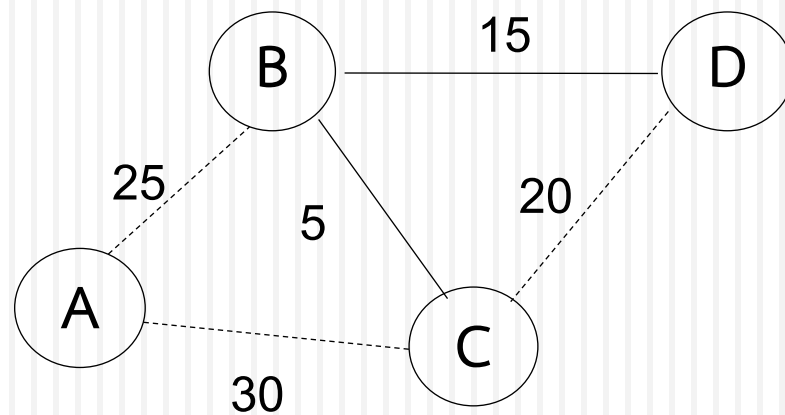
2. เลือก (BC, 5) ออกจากคิว เป็นเส้นทางจริง



$PQ = \{(BD, 15), (CD, 20), (AB, 25), (AC, 30)\}$

อัลกอริทึมของครุสคัล (Kruskal's Algorithm)

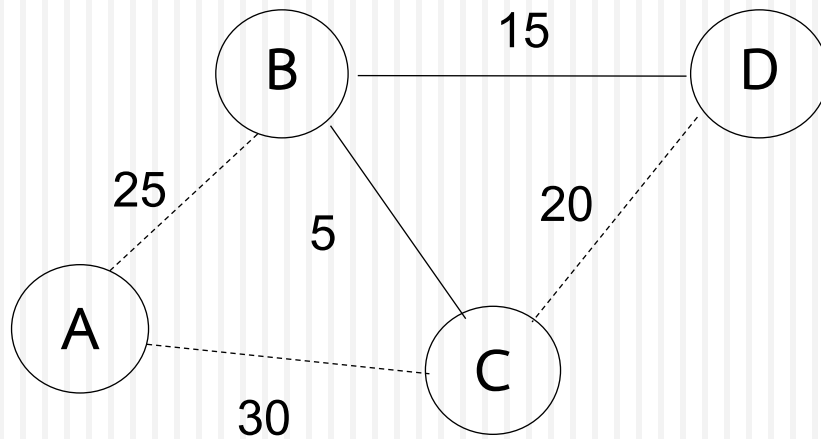
3. เลือก (BD, 15) ออกจากคิว เป็นเส้นทางจริง



$$PQ = \{(CD, 20), (AB, 25), (AC, 30)\}$$

อัลกอริทึมของครุสคัล (Kruskal's Algorithm)

4. เลือก (BD, 15) ออกจากคิว เป็นเส้นทางจริง



$$PQ = \{(CD, 20), (AB, 25), (AC, 30)\}$$

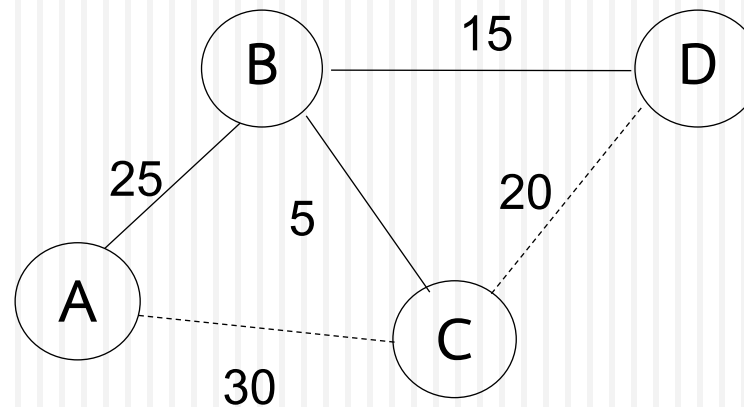
อัลกอริธึมของครุสคัล (Kruskal's Algorithm)

5. เลือก (CD,20) ออกจากคิว เป็นเส้นทางจริง แต่เนื่องจากเมื่อเลือก CD เป็นเส้นทางจริง จะทำให้เกิดวงจรปิดขึ้น ให้ยกเลิกเส้นทาง CD

$PQ = \{(AB, 25), (AC, 30)\}$

6. เลือก (AB, 25) ออกจากคิว เป็นเส้นทางจริง

$PQ = \{(AC, 30)\}$



7. เลือก (AC, 30) ออกจากคิว เป็นเส้นทางจริง แต่เนื่องจากเมื่อเลือก AC เป็นเส้นทางจริง จะทำให้เกิดวงจรปิดขึ้น ให้ยกเลิกเส้นทาง AC

อัลกอริธึมของพริม (Prim's Algorithm)

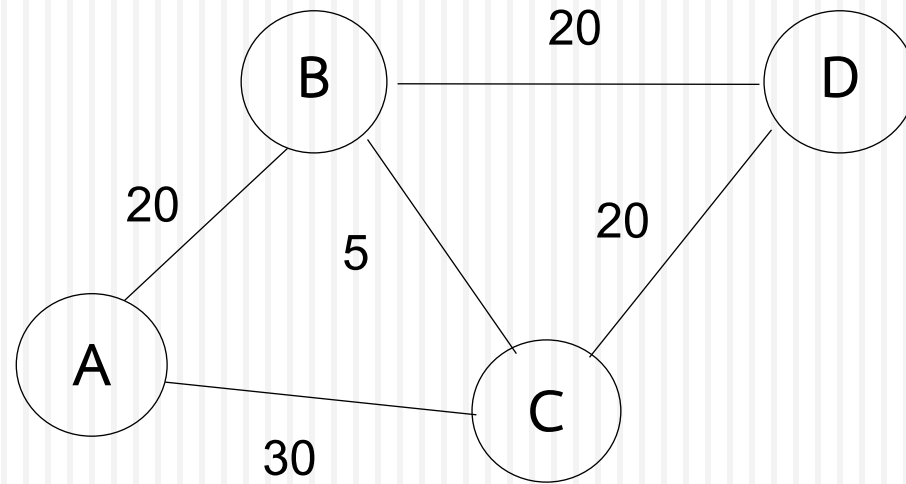
อัลกอริธึมของพริม มีลักษณะคล้ายกับการหาเส้นทางที่สั้นที่สุดของ Dijkstra ซึ่งจะเริ่มต้นจากจุดยอดที่ต้องการแล้วหาค่าน้ำหนักไปตามจุดยอดที่อยู่ติดกัน แล้วเก็บลงไปในคิวแบบมีความสำคัญ โดยความสำคัญคือค่าน้ำหนักที่มีค่าน้อยคือมีความสำคัญมาก แล้วเลือกจุดยอดที่มีค่าน้ำหนักน้อยสุดออกมาจากคิว แล้วหาจุดยอดที่ติดกันซ้ำอย่างนี้ไปเรื่อยๆ จนครบทุกจุดยอด แต่จะไม่มีมีการปรับค่าน้ำหนักประจำจุดยอดเหมือนกับวิธีของ Dijkstra

อัลกอริธึมของพริม

1. PriorityQueue = {startVertex}
2. ขณะที่ PriorityQueue ไม่ว่าง ทำ
 - 2.1 ดึงสมาชิกตัวแรกใน Queue ออกมา(ให้ชื่อว่า X)
 - 2.2 หาก X เคยถูกเลือกเป็นเส้นทางแล้ว กลับไปข้อ 1.1
 - 2.3 เลือกโหนด X ให้เป็นเส้นทางจริง
 - 2.4 สำหรับโหนดใดๆ ที่เชื่อมต่อกับ X ให้ทำดังนี้
 - 2.4.1 นำทุกเส้นทางไปไว้ใน PriorityQueue

ตัวอย่างอัลกอริธึมของพริม

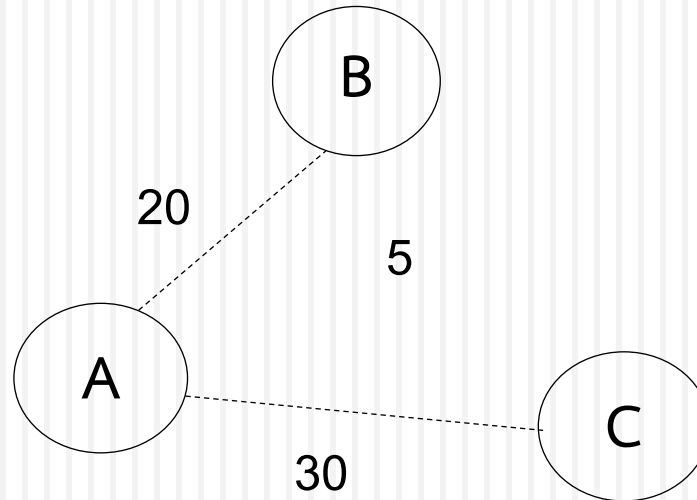
1. กำหนดจุดยอดเริ่มต้นที่จุดยอด A



ตัวอย่างอัลกอริธึมของพริม

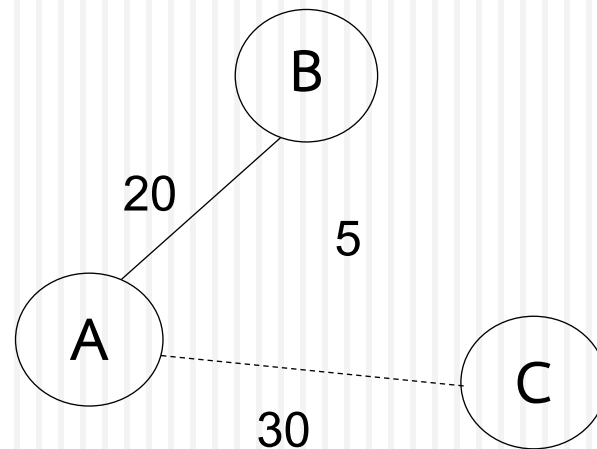
2. หาเส้นทางจากจุดยอด A ไปยังจุดยอดอื่นๆ แล้วเก็บไว้ในคิวแบบมีลำดับความสำคัญ

$PQ = \{(AB, 20), (AC, 30)\}$



ตัวอย่างอัลกอริทึมของพริม

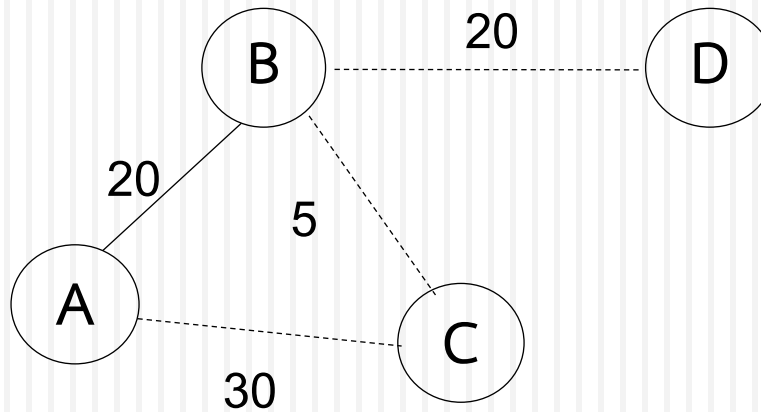
3. นำจุดยอด B จากคิวออกมาเลือกเป็นเส้นทางจริง



ตัวอย่างอัลกอริธึมของพริม

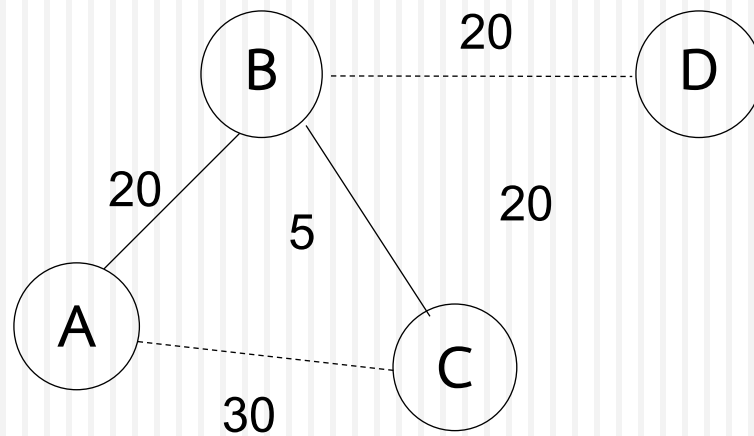
4. หาเส้นทางจากจุดยอด B ไปยังจุดยอดอื่นๆ แล้วเก็บไว้ในคิวแบบมีลำดับความสำคัญ

$$PQ = \{(BC, 5), (BD, 20), (AC, 30)\}$$



ตัวอย่างอัลกอริธึมของพริม

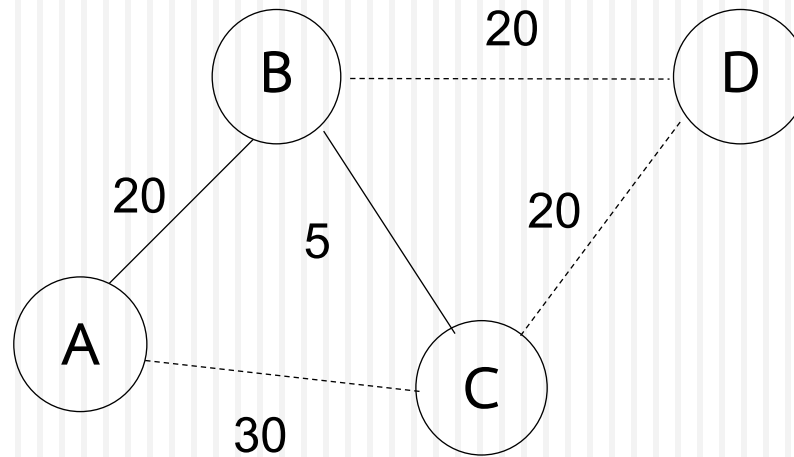
5. นำจุดยอด C จากคิวนอกมาเลือกเป็นเส้นทางจริง



ตัวอย่างอัลกอริธึมของพริม

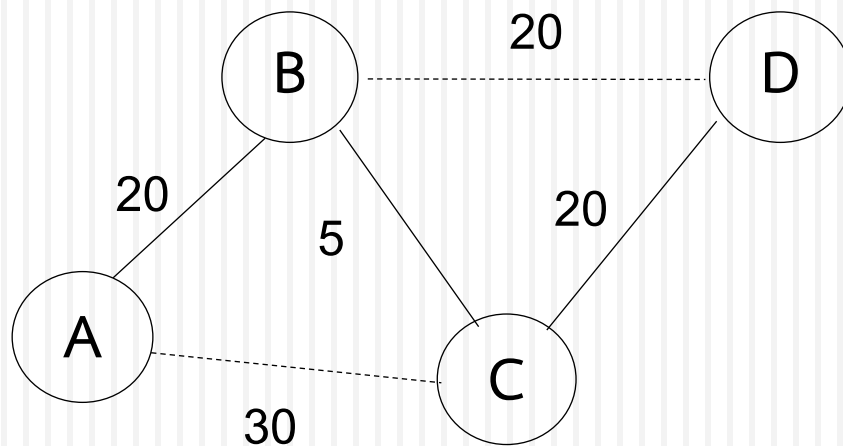
6. หาเส้นทางจากจุดยอด C ไปยังจุดยอดอื่นๆ แล้วเก็บไว้ในคิวแบบมีลำดับความสำคัญ

$$PQ = \{(CD, 20), (BD, 20), (AC, 30)\}$$



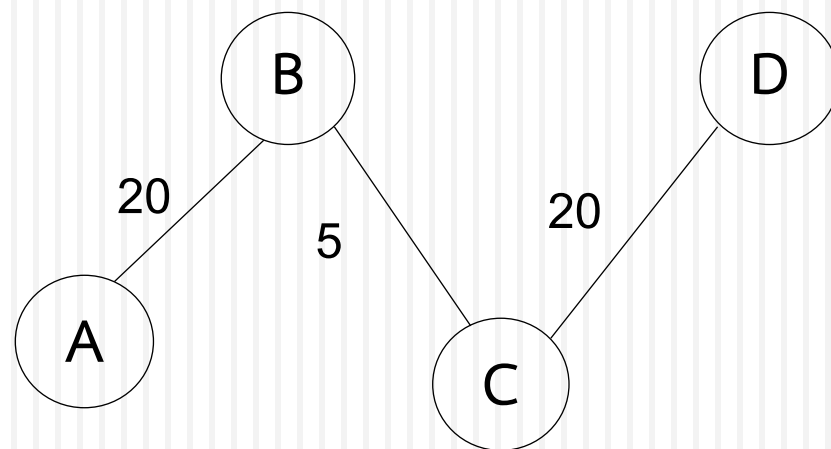
ตัวอย่างอัลกอริทึมของพริม

7. นำจุดยอด C จากคิวนอกมาเลือกเป็นเส้นทางจริง



ตัวอย่างอัลกอริธึมของพริม

8. เส้นทางอื่นๆ ที่ไม่ได้ถูกเลือกจะถูกทิ้งไป ได้เป็นต้นไม้ทอดข้ามน้อยที่สุด



การหาเส้นทางที่สั้นที่สุด (Shortest Path Algorithm)

อัลกอริธึมของไดค์สตรา (Dijkstra's algorithm) ซึ่งคิดโดยนาย Edsger Dijkstra เป็นอัลกอริธึมที่ทำการค้นหาเส้นทางที่สั้นที่สุด (Shortest path) ที่จะใช้เดินทางจากจุดเริ่มต้นจุดหนึ่ง ไปยังจุดสิ้นสุดใดๆ บนกราฟ โดยผ่านเส้นทางที่เชื่อมตรง (Edge) ระหว่างหน่วยหรือจุดต่างๆ (Node) อัลกอริธึมนี้ มีชื่อที่นิยมเรียกอีกชื่อหนึ่งคือ อัลกอริธึมค้นหาเส้นทางที่สั้นที่สุด (Shortest Path Algorithm) ซึ่งจะประยุกต์ใช้กราฟ ในการหาเส้นทางที่สั้นที่สุด มีประโยชน์ในการหาเส้นทาง ขนส่งสินค้า หรือในปัญหาการเดินทางของเซลส์แมน (Travelling Salesman Problem)

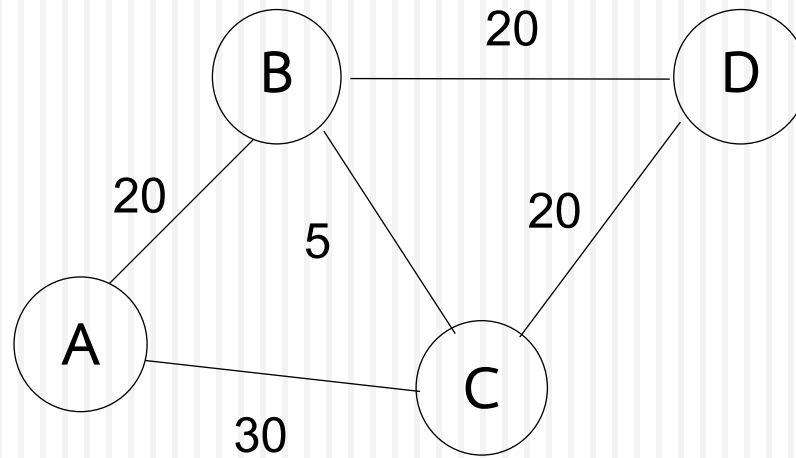
อัลกอริธึมการหาเส้นทางที่สั้นที่สุด

อัลกอริธึม ของ Dijkstra คล้ายกับ Breadth-first Search แต่จะเปลี่ยนเป็นคิวแบบมีลำดับความสำคัญโดยน้ำหนักที่นำมาใช้ในการพิจารณาความสำคัญ ใช้น้ำหนักของเส้นทางรวมจากจุดเริ่มต้นมายังโหนดนั้นๆ อัลกอริธึมนี้ใช้ได้กับขอบที่มีค่าตั้งแต่ 0 ขึ้นไป

1. PriorityQueue = {startVertex}
2. ขณะที่ PriorityQueue ไม่ว่าง ทำ
 - 2.1 ดึงสมาชิกตัวแรกใน Queue ออกมา(ให้ชื่อว่า X)
 - 2.2 หาก X เคยถูกเลือกเป็นเส้นทางแล้ว กลับไปข้อ 2.1
 - 2.3 เลือกโหนด X ให้เป็นเส้นทางจริง
 - 2.4 สำหรับโหนดใดๆ ที่เชื่อมต่อกับ X ให้ทำดังนี้
 - 2.4.1 คำนวณระยะทางรวมของโหนด X มายังโหนดนั้นๆ
 - 2.4.2 นำทุกเส้นทางไปไว้ใน PriorityQueue

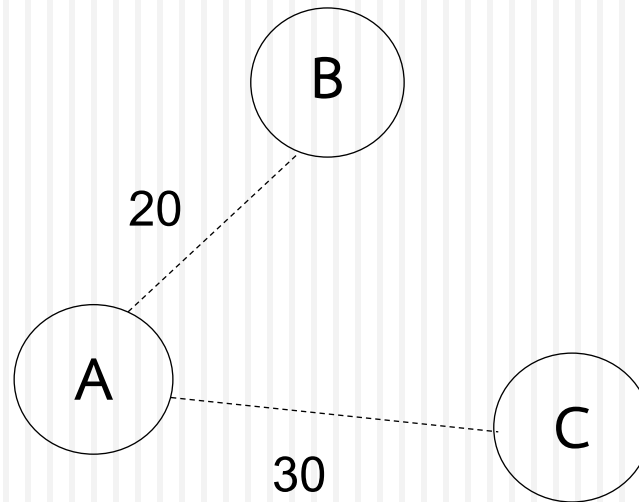
ตัวอย่างการหาเส้นทางที่สั้นที่สุด

1. $PQ = \{ (A,0) \}$



ตัวอย่างการหาเส้นทางที่สั้นที่สุด

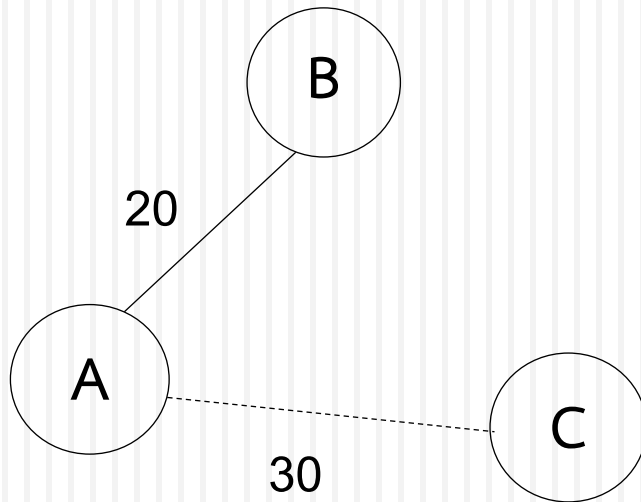
2. เลือก (AA,0) เป็นเส้นทางจริง แล้วคำนวณหาเส้นทางรวมจาก A ไปยัง จุดยอดอื่นๆ แล้วเก็บลงไปในคิวแบบอภิสีทธิ์ตามลำดับความสำคัญ



ตัวอย่างการหาเส้นทางที่สั้นที่สุด

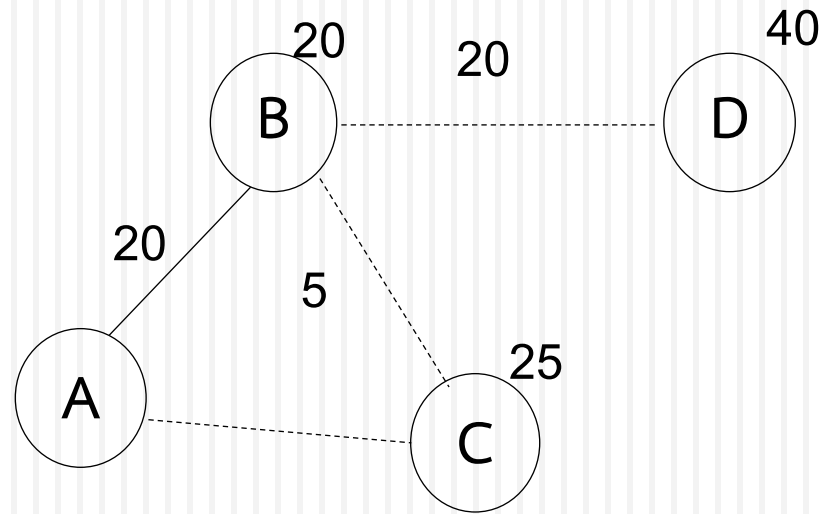
3. $PQ = \{(AB,20),(AC,30)\}$

4. เลือก $(AB,20)$ เป็นเส้นทางจริง แล้วคำนวณหาเส้นทางรวมจาก B ไปยัง จุดยอดอื่นๆ แล้วเก็บลงไปในคิวแบบอภิสีทธิ์ตามลำดับความสำคัญ



ตัวอย่างการหาเส้นทางที่สั้นที่สุด

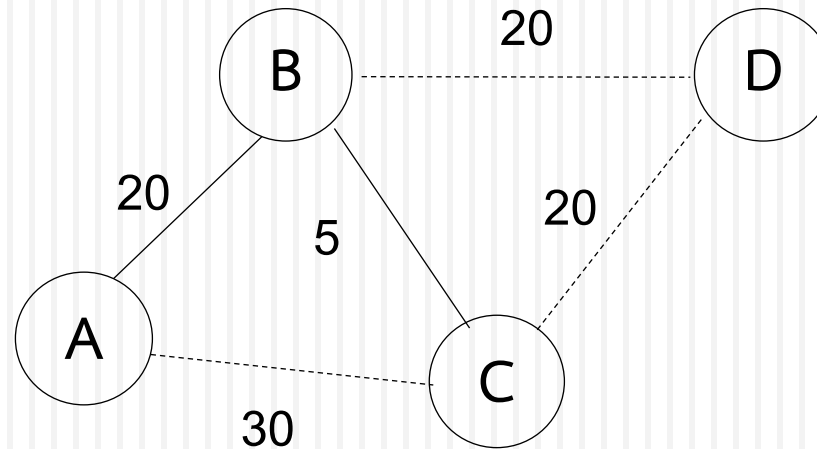
5. $PQ = \{(BC,25),(AC,30),(BD,40)\}$



ตัวอย่างการหาเส้นทางที่สั้นที่สุด

6. เลือก (BC,25) เป็นเส้นทางจริง แล้วคำนวณหาเส้นทางรวมจาก C ไปยัง จุดยอดอื่นๆ แล้วเก็บลงไปในคิวแบบอภิสีทธิ์ตามลำดับความสำคัญ

7. $PQ = \{(AC,30),(BD,40),(CD,45)\}$

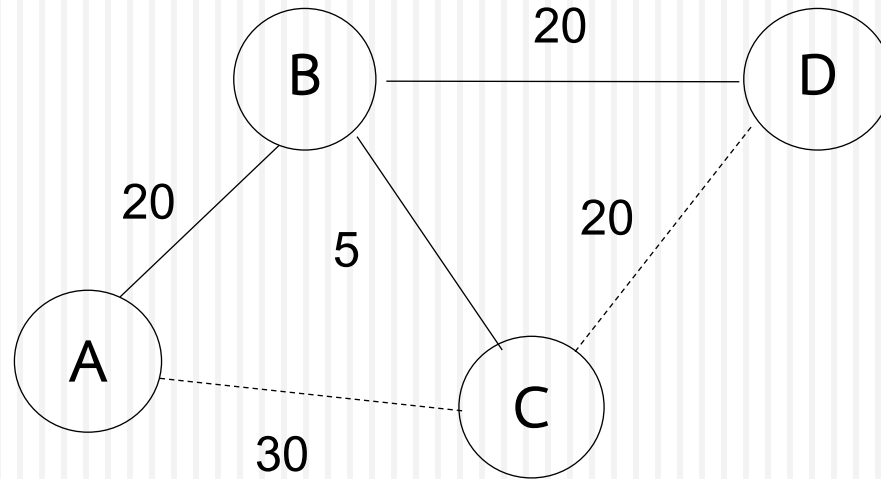


ตัวอย่างการหาเส้นทางที่สั้นที่สุด

8. ดึง AC ออกมาจากคิว เพื่อเลือกเป็นเส้นทางจริง แต่เนื่องจาก C ถูกเลือกเป็นเส้นทางจริงแล้วจึงไม่นำมาสนใจ

$$PQ = \{(BD,40),(CD,45)\}$$

9. ดึง (BD,40) ออกมาจากคิว เลือก D เป็นเส้นทางจริง

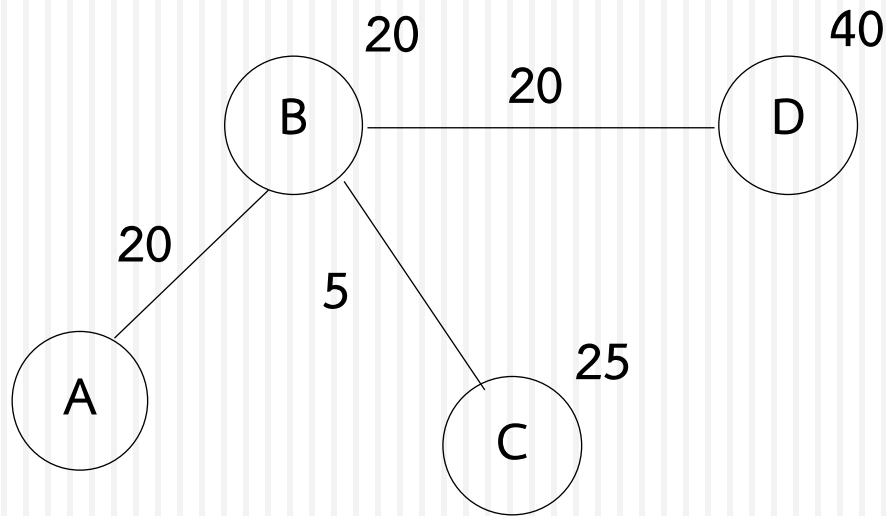


ตัวอย่างการหาเส้นทางที่สั้นที่สุด

10. $PQ = \{(CD, 45)\}$

11. ดึง $(CD, 45)$ ออกจากคิว เพื่อเลือกเป็นเส้นทางจริง แต่เนื่องจาก D ถูกเลือกเป็นเส้นทางจริงแล้ว จึงไม่นำมาสนใจ

12. เมื่อคิวว่างจึงจบการทำงาน





Q & A